



Parul University

FACULTY OF ENGINEERING AND TECHNOLOGY BACHELOR OF TECHNOLOGY

**CRYPTOGRAPHY
303105266**

4th Semester

COMPUTER SCIENCE AND ENGINEERING DEPARTMENT

**LAB MANUAL
SESSION: 2025-26**

CERTIFICATE

This is to certify that **Mr. PATEL KRISH NITIN** with Enrollment no. **2403031260022** has successfully completed his laboratory work in **CRYPTOGRAPHY (303105266)** from the department of **COMPUTER SCIENCE & ENGINEERING** during the academic year **2025-2026**.



Date of Submission: _____ Staff in charge: _____

Head of Department: _____

Practical Assessment Table

Sr. No.	Experiment Title	Date	Page No.		Marks (10)	Sign
			From	To		
1	1.1 Implement Caesar Cipher and apply brute-force attack. 1.2 Cryptanalysis attacks on Playfair Cipher (Key: PARUL).					
2	1: Statistical Frequency Analyzer Features: • Single-letter, digraph, trigram frequency • Comparison with standard English frequencies • Predicted plaintext mapping • Frequency table + graph 2: Secure Monoalphabetic Cipher Generator. Features: • Generates a random valid monoalphabetic key (no repeats) • Encrypts plaintext using the generated key • Calculates Index of Coincidence (IC) • Explains whether frequency analysis will be effective 3: Monoalphabetic Cipher Breaking Tool (Cryptanalysis). Features: • Takes ciphertext as input • Uses frequency analysis to suggest substitutions • Allows interactive key refinement • Displays partially decrypted text after each refinement					
3	Write a program to implement DES encryption and decryption strategy by taking arguments from user and perform 16 rounds of iterations to get cipher text.					
4	Implement RSA cryptosystem.					

5	Implement Message Integrity using SHA-256 hashing function which creates chain of three blocks. Each block contains index, timestamp, data, previous hash value and current block of hash value. Test message integrity of program by modifying one of the hash value of block.					
6	Implement Digital Signature.					
7	Study and Configure SSH to authenticate machines, generate session key and transfer files using symmetric key and asymmetric key cryptographic system.					
8	Implement Deffi Hellmn key exchange algorithm. Generate share secret without Sharing the secret code.					
9	Implement Elgamal Cryptosystem.					
10	Prepare report on any advanced Cryptographic system.					

PRACTICAL – 1

Aim1.1: Implement Caesar Cipher and apply brute-force attack.

Definition: To implement the Caesar Cipher technique for encryption and decryption using Python and apply a brute-force attack to retrieve the original key.

Theory: Caesar Cipher is one of the earliest known substitution ciphers. Each letter in the plaintext is shifted by a fixed number of positions (key) down the alphabet.

Example:

Plaintext: HELLO

Key: 3

Ciphertext: KHOOR

Encryption Rule: $C = (P+K) \bmod 26$

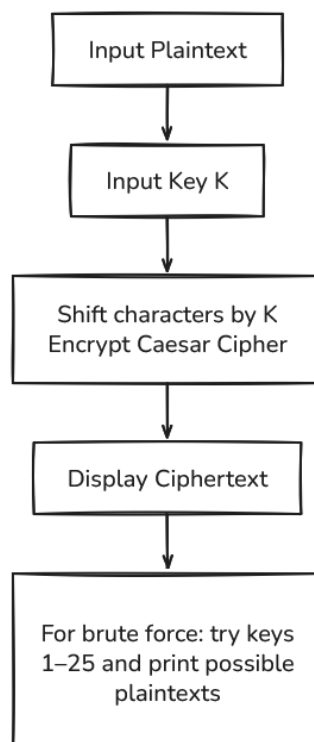
Decryption Rule: $P = (C-K) \bmod 26$

Brute Force Attack: Since the Caesar Cipher has only 26 possible keys, the attacker tries all keys from 1 to 25 to check which plaintext makes sense.

Algorithm:

1. Take plaintext and key.
2. For encryption, shift characters by key.
3. For decryption, subtract key.
4. Brute Force: Try key = 1 to 25 and display outputs.

Flowchart:



Code:

```
def caesar_cipher(text: str, shift: int) -> str:
    shift = shift % 26
    result = []
    for char in text:
```

```
def caesar_cipher(char, shift):
    if char.isalpha():
        base = ord('A') if char.isupper() else ord('a')
        shifted = (ord(char) - base + shift) % 26
        result.append(chr(base + shifted))
    else:
        result.append(char)
return "".join(result)

def brute_force_attack(ciphertext: str):
    print("\n--- Attempting Brute Force Attack ---")
    for shift in range(1, 26):
        decrypted = caesar_cipher(ciphertext, 26 - shift)
        print(f"Key {shift:2}: {decrypted}")

original_text = input("Enter text to encrypt: ")
key = int(input("Enter key (1-25): "))
encrypted = caesar_cipher(original_text, key)
print(f"Encrypted: {encrypted}")
brute_force_attack(encrypted)
```

Input:

Enter text to encrypt: Ankur
Enter key (1-25): 17

Output:

Encrypted: Rebli
--- Attempting Brute Force Attack ---
Key 1: Qdakh
Key 2: Pczjg
Key 3: Obyif
Key 4: Naxhe
Key 5: Mzwgd
Key 6: Lyvfc
Key 7: Kxueb
Key 8: Jwtda
Key 9: Ivscz
Key 10: Hurby
Key 11: Gtqax
Key 12: Fspzw
Key 13: Eroyv
Key 14: Dqnxu
Key 15: Cpmwt
Key 16: Bolvs
Key 17: Ankur
Key 18: Zmjtg
Key 19: Ylisp
Key 20: Xkhro
Key 21: Wjgqn
Key 22: Vifpm
Key 23: Uheol
Key 24: Tgdnk
Key 25: Sfcmj

Conclusion: Caesar Cipher is highly insecure due to its small keyspace (only 26 possible keys). A brute-force attack is enough to break the cipher instantly.

Aim1.2: Cryptanalysis attacks on Playfair Cipher (Key: PARUL).

Description: To apply cryptanalysis techniques to decrypt the original message from a given ciphertext using the playfair cipher, using the key = PARUL.

Theory: Playfair Cipher is a digraph substitution cipher invented by Charles Wheatstone in 1854. It encrypts pairs of letters (digraphs) instead of single characters.

Steps in PlayFair Cipher:

STEP 1: Generate a 5×5 Key Matrix.

Rules: 1. Write the key without repeating letters.

2. Continue filling the matrix with the remaining letters of the alphabet.

3. Combine I/J in one cell.

4. Matrix is filled row wise.

For Key = PARUL,

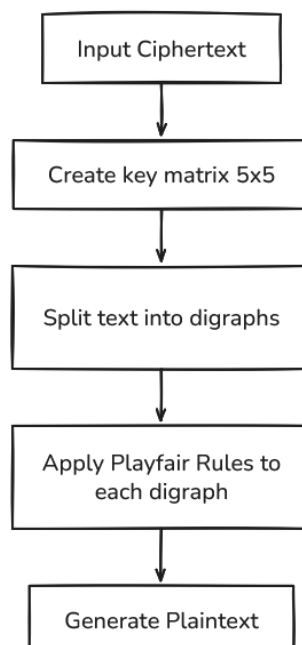
P	A	R	U	L
B	C	D	E	F
G	H	I/J	K	M
N	O	Q	S	T
V	W	X	Y	Z

STEP 2: Preprocess Ciphertext.

- Split into pairs of 2 letters (digraphs).
- If both letters are same, insert filler X.
- If the resulting string is odd length, add X at the end.

STEP 3: Encryption Rules

- Rule 1: Same Row, take letters to the immediate left.
- Rule 2: Same column, take letters immediately above.
- Rule 3: Rectangle rule, Form a rectangle and take letters at the opposite corners in the same row.

Flowchart:

Code:

```
import string
def generate_key_matrix(key):
    key = key.upper().replace("J", "I")
    matrix = []
    used = set()
    for char in key:
        if char not in used and char.isalpha():
            matrix.append(char)
            used.add(char)
    for char in string.ascii_uppercase:
        if char not in used and char != "J":
            matrix.append(char)
            used.add(char)
    key_matrix = [matrix[i:i+5] for i in range(0, 25, 5)]
    return key_matrix
def find_position(matrix, letter):
    for row in range(5):
        for col in range(5):
            if matrix[row][col] == letter:
                return row, col
    return None
def decrypt_playfair(cipher, matrix):
    cipher = cipher.upper().replace("J", "I")
    cipher = cipher.replace(" ", "")
    if len(cipher) % 2 != 0:
        cipher += "X"
    plaintext = ""
    for i in range(0, len(cipher), 2):
        a = cipher[i]
        b = cipher[i+1]
        row1, col1 = find_position(matrix, a)
        row2, col2 = find_position(matrix, b)
        if row1 == row2:
            plaintext += matrix[row1][(col1 - 1) % 5]
            plaintext += matrix[row2][(col2 - 1) % 5]
        elif col1 == col2:
            plaintext += matrix[(row1 - 1) % 5][col1]
            plaintext += matrix[(row2 - 1) % 5][col2]
        else:
            plaintext += matrix[row1][col2]
            plaintext += matrix[row2][col1]
    return plaintext
key = "PARUL"
cipher_text = input("Enter Playfair cipher text: ")
matrix = generate_key_matrix(key)
print("\nKey Matrix:")
for row in matrix:
    print(row)
decrypted = decrypt_playfair(cipher_text, matrix)
print("\nDecrypted Text:", decrypted)
```


Input:

Enter Playfair cipher text: L00LEH

Output:

Key Matrix:

```
['P', 'A', 'R', 'U', 'L']  
['B', 'C', 'D', 'E', 'F']  
['G', 'H', 'I', 'K', 'M']  
['N', 'O', 'Q', 'S', 'T']  
['V', 'W', 'X', 'Y', 'Z']
```

Decrypted Text: ATTACK

Conclusion: The Playfair Cipher increases security over monoalphabetic ciphers by encrypting digraphs. However, it is still vulnerable to frequency analysis of digraphs, crib attacks, and known-plaintext attacks.

PRACTICAL – 2

Aim 1: Statistical Frequency Analyzer

Features:

- **Single-letter, digraph, trigraph frequency**
- **Comparison with standard English frequencies**
- **Predicted plaintext mapping**
- **Frequency table + graph**

Description: This experiment implements a cryptanalysis tool that performs N-gram frequency analysis on ciphertext. It identifies the statistical distribution of single letters, digraphs, and trigraphs to predict the underlying plaintext by comparing cipher statistics with standard English language patterns.

Theory: Statistical frequency analysis is a classical cryptanalysis technique used to break substitution ciphers. It relies on the fact that natural languages exhibit non-uniform letter distributions. In English, certain letters (such as *E, T, A, O*) appear more frequently than others.

By analyzing:

- Monographs (single letters)
- Digraphs (pairs of letters)
- Trigraphs (triplets of letters)

an attacker can infer likely plaintext substitutions from ciphertext patterns.

Comparing observed frequencies with known English statistics allows prediction of plaintext characters. Visualization using tables and graphs improves clarity and pattern recognition.

This method is especially effective against monoalphabetic substitution ciphers, as encryption preserves statistical structure.

Mathematical Concepts:

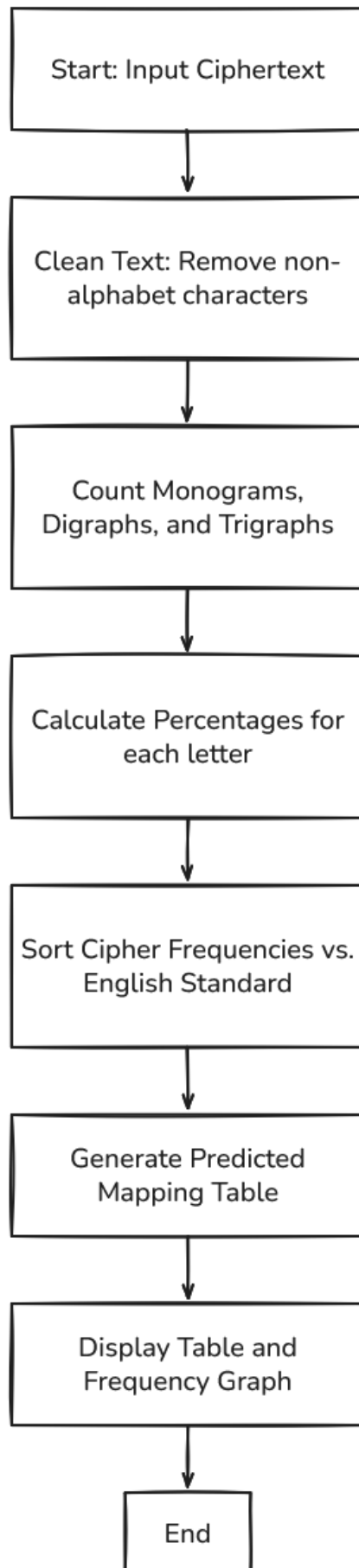
1. Frequency Calculation

$$\left[f(x) = \frac{\text{Count of } x}{\text{Total characters}} \right]$$

2. Digraph / Trigraph Frequency

$$\left[f(xy) = \frac{\text{Count of digraph } xy}{\text{Total digraphs}} \right]$$

3. Statistical Comparison: Observed frequency distribution is compared with standard English frequencies using ranking or visual alignment.
4. Graphical Representation
 - Bar charts visualize deviation from expected frequencies.
 - Peaks indicate high-probability plaintext mappings.

Flowchart:

Code:

```
import string
from collections import Counter
import matplotlib.pyplot as plt
ENGLISH_FREQ = {
    'A': 8.17, 'B': 1.49, 'C': 2.78, 'D': 4.25, 'E': 12.70,
    'F': 2.23, 'G': 2.02, 'H': 6.09, 'I': 6.97, 'J': 0.15,
    'K': 0.77, 'L': 4.03, 'M': 2.41, 'N': 6.75, 'O': 7.51,
    'P': 1.93, 'Q': 0.10, 'R': 5.99, 'S': 6.33, 'T': 9.06,
    'U': 2.76, 'V': 0.98, 'W': 2.36, 'X': 0.15, 'Y': 1.97,
    'Z': 0.07
}

def clean_text(text):
    return ''.join(c for c in text.upper() if c in string.ascii_uppercase)

def ngram_frequency(text, n):
    return Counter(text[i:i+n] for i in range(len(text) - n + 1))

def frequency_analyzer(ciphertext):
    text = clean_text(ciphertext)
    mono_freq = Counter(text)
    total = sum(mono_freq.values())
    mono_percent = {k: (v / total) * 100 for k, v in mono_freq.items()}
    di_freq = ngram_frequency(text, 2)
    tri_freq = ngram_frequency(text, 3)
    sorted_cipher = sorted(mono_percent.items(), key=lambda x: x[1], reverse=True)
    sorted_english = sorted(ENGLISH_FREQ.items(), key=lambda x: x[1], reverse=True)
    predicted_mapping = {
        cipher[0]: english[0]
        for cipher, english in zip(sorted_cipher, sorted_english)
    }
    return mono_percent, di_freq, tri_freq, predicted_mapping

def display_results(mono, di, tri, mapping):
    print("\n--- Single Letter Frequency (%) ---")
    for k, v in sorted(mono.items()):
        print(f"{k}: {v:.2f}")
    print("\n--- Top 10 Digraphs ---")
    for k, v in di.most_common(10):
        print(f"{k}: {v}")
    print("\n--- Top 10 Trigraphs ---")
    for k, v in tri.most_common(10):
        print(f"{k}: {v}")

    print("\n--- Predicted Plaintext Mapping ---")
    for k, v in mapping.items():
        print(f"{k} → {v}")

def plot_frequency(mono):
    letters = list(string.ascii_uppercase)
    values = [mono.get(l, 0) for l in letters]
    plt.bar(letters, values)
    plt.xlabel("Letters")
    plt.ylabel("Frequency (%)")
    plt.title("Ciphertext Letter Frequency")
    plt.show()

if __name__ == "__main__":
    ciphertext = input("Enter Ciphertext for Frequency Analysis:\n")
```

```
mono, di, tri, mapping = frequency_analyzer(ciphertext)
display_results(mono, di, tri, mapping)
plot_frequency(mono)
```

Input:

Enter Ciphertext for Frequency Analysis:

L00LEH

Output:

--- Single Letter Frequency (%) ---

E: 16.67

H: 16.67

L: 33.33

O: 33.33

--- Top 10 Digraphs ---

LO: 1

OO: 1

OL: 1

LE: 1

EH: 1

--- Top 10 Trigraphs ---

L00: 1

00L: 1

OLE: 1

LEH: 1

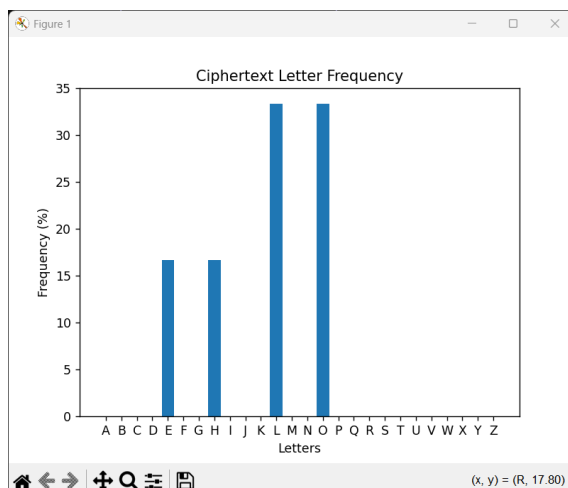
--- Predicted Plaintext Mapping ---

L → E

O → T

E → A

H → O



Conclusion: The experiment shows that ciphertext preserves language patterns. By using frequency analysis and graphs, plaintext letters can be predicted easily, proving monoalphabetic ciphers are weak against statistical attacks.

Aim 2: Secure Monoalphabetic Cipher Generator

Features:

- * Generates a random valid monoalphabetic key (no repeats)
- * Encrypts plaintext using the generated key
- * Calculates Index of Coincidence (IC)
- * Explains whether frequency analysis will be effective

Description: This experiment involves generating a random monoalphabetic substitution key in which each plaintext letter is mapped to a unique ciphertext letter without repetition. Using this key, the given plaintext is encrypted to produce ciphertext. The experiment also calculates the Index of Coincidence (IC) of the ciphertext to analyze its statistical properties. Based on the IC value, the effectiveness of frequency analysis is evaluated, helping to understand why monoalphabetic ciphers remain vulnerable despite having a large keyspace.

Theory: A monoalphabetic cipher replaces each plaintext letter with a unique ciphertext letter using a fixed substitution key. Security depends on the secrecy of the key, which is a permutation of the alphabet.

Although brute force is impractical ($26!$ keys), monoalphabetic ciphers remain vulnerable to frequency analysis, since substitution does not alter letter frequency patterns.

The Index of Coincidence (IC) measures how closely the ciphertext resembles a natural language.

Mathematical Concepts:

1. Key Space

$$|\text{Keyspace}| = 26! \approx 4 \times 10^{26}$$

2. Encryption Function

$$C = K(P)$$

Where:

- (P) = plaintext letter
- (K) = substitution mapping
- (C) = ciphertext letter

3. Index of Coincidence

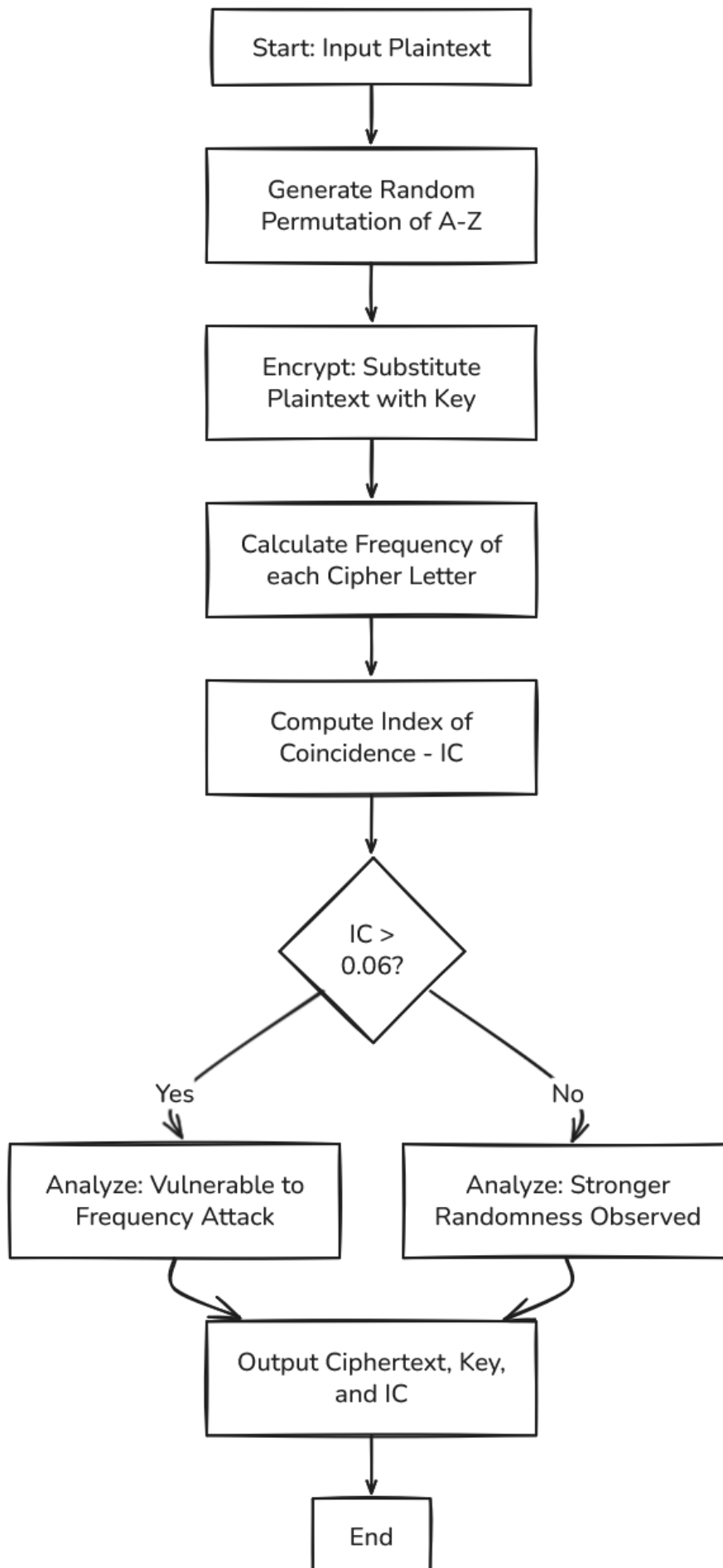
$$IC = \frac{\sum f_i(f_i - 1)}{N(N - 1)}$$

Where:

- (f_i) = frequency of letter (i)
- (N) = total letters

4. IC Interpretation

IC Value	Meaning
≈ 0.065	English plaintext
≈ 0.038	Random text
≈ 0.06	Monoalphabetic cipher

Flowchart:

Code:

```
import string
import random
from collections import Counter

def clean_text(text):
    return ''.join(c for c in text.upper() if c in string.ascii_uppercase)

def generate_monoalphabetic_key():
    letters = list(string.ascii_uppercase)
    shuffled = letters.copy()
    random.shuffle(shuffled)
    return dict(zip(letters, shuffled))

def encrypt(plaintext, key):
    plaintext = clean_text(plaintext)
    return ''.join(key[c] for c in plaintext)

def index_of_coincidence(text):
    text = clean_text(text)
    freq = Counter(text)
    N = len(text)
    if N <= 1:
        return 0.0
    ic = sum(v * (v - 1) for v in freq.values()) / (N * (N - 1))
    return ic

def analyze_frequency_attack(ic):
    if ic > 0.06:
        return "Frequency analysis is likely to be effective (text resembles English).\"
    elif ic < 0.04:
        return "Frequency analysis will be difficult (text appears random).\"
    else:
        return "Frequency analysis may be partially effective.\"

def display_key(key):
    print("\n--- Monoalphabetic Key ---")
    for k in key:
        print(f"{k} → {key[k]}")

if __name__ == "__main__":
    plaintext = input("Enter Plaintext to Encrypt -> ")
    key = generate_monoalphabetic_key()
    ciphertext = encrypt(plaintext, key)
    ic = index_of_coincidence(ciphertext)
    explanation = analyze_frequency_attack(ic)
    display_key(key)
    print("\nPlaintext :", plaintext)
    print("Ciphertext:", ciphertext)
    print(f"\nIndex of Coincidence (IC): {ic:.4f}")
    print("Analysis:", explanation)
```


Input:

Enter Plaintext to Encrypt -> P@rul#2k26

Output:

--- Monoalphabetic Key ---

A → X
B → Z
C → J
D → E
E → F
F → M
G → L
H → K
I → R
J → W
K → Y
L → Q
M → A
N → B
O → P
P → T
Q → N
R → H
S → G
T → O
U → S
V → I
W → V
X → U
Y → D
Z → C

Plaintext : P@rul#2k26

Ciphertext: THSQY

Index of Coincidence (IC): 0.0000

Analysis: Frequency analysis will be difficult (text appears random).

Conclusion: The experiment proves that even when a random and valid monoalphabetic key is used, the cipher is not secure against statistical attacks. The calculated Index of Coincidence remains close to that of English text, confirming that frequency analysis can still be applied effectively. This highlights that a large keyspace alone does not ensure strong security.

Aim 3: Monoalphabetic Cipher Breaking Tool (Cryptanalysis)

Features:

- * Takes ciphertext as input
- * Uses frequency analysis to suggest substitutions
- * Allows interactive key refinement
- * Displays partially decrypted text after each refinement

Description: This experiment focuses on breaking a monoalphabetic substitution cipher using cryptanalysis techniques. The ciphertext is analyzed to compute letter frequencies, which are then used to suggest possible plaintext substitutions. The tool allows interactive refinement of the substitution key, and after each change, a partially decrypted text is displayed. This step-by-step process demonstrates how frequency analysis and logical guessing can be combined to successfully recover the original plaintext.

Theory: Cryptanalysis of monoalphabetic ciphers exploits the preserved statistical structure of the plaintext. Frequency analysis provides an initial key guess by mapping frequent ciphertext letters to frequent English letters.

Due to ambiguities, human intuition and iterative refinement are required. Interactive tools allow attackers to:

- Modify substitutions
- Observe partially decrypted text
- Gradually converge to correct plaintext

This mimics real-world classical cryptanalysis methods.

Mathematical Concepts:

1. Frequency Ranking

Ciphertext letters ranked by:

$$f_1 > f_2 > \dots > f_{26}$$

Mapped to English frequency order:

$$E > T > A > O > I > N > \dots$$

2. Partial Decryption Function

$$P' = D(C, K')$$

Where:

- (K') = partial or refined key

3. Iterative Improvement

Each refinement increases plaintext readability, reducing entropy.

Code:

```
import string
from collections import Counter
ENGLISH_ORDER = "ETAOINSHRDLUMWFGYPBVKJXQZ"
def clean_text(text):
    return ''.join(c.upper() if c.upper() in string.ascii_uppercase else ' ' for c in text)
def frequency_analysis(text):
    text = clean_text(text)
    freq = Counter(c for c in text if c in string.ascii_uppercase)
    return ''.join(c for c, _ in freq.most_common())
def initial_key(ciphertext):
    cipher_order = frequency_analysis(ciphertext)
    key = {}
```

```
for c, e in zip(cipher_order, ENGLISH_ORDER):
    key[c] = e
return key
def decrypt(ciphertext, key):
    result = ""
    for c in ciphertext.upper():
        if c in key:
            result += key[c]
        elif c in string.ascii_uppercase:
            result += "_"
        else:
            result += c
    return result
def print_key(key):
    print("\nCurrent Key:")
    for c in sorted(key):
        print(f"{c} → {key[c]}")
def interactive_cryptanalysis(ciphertext):
    key = initial_key(ciphertext)
    while True:
        print("\n-----")
        print("Partially Decrypted Text:\n")
        print(decrypt(ciphertext, key))
        print_key(key)
        choice = input("\nSwap plaintext letters (A B) or type 'exit': ").upper()
        if choice == "EXIT":
            break
        try:
            a, b = choice.split()
            for k in key:
                if key[k] == a:
                    key[k] = b
                elif key[k] == b:
                    key[k] = a
        except:
            print("Invalid input. Example: E T")

if __name__ == "__main__":
    ciphertext = ""
    ZIT JXXK GKZXPX ZKZXLKJGZXKJ GXVRKX
    ""
    interactive_cryptanalysis(ciphertext)
```

Output:

```
-----
Partially Decrypted Text:
  ANS OETE ITAEHE ATAERTOIAETO IEDLTE
```

Current Key:

G → I

I → N

J → O
K → T
L → R
P → H
R → L
T → S
V → D
X → E
Z → A

Swap plaintext letters (A B) or type 'exit': E R

Partially Decrypted Text:

ANS ORTR ITARHR ATARETOIARTO IRDLTR

Current Key:

G → I
I → N
J → O
K → T
L → E
P → H
R → L
T → S
V → D
X → R
Z → A

Swap plaintext letters (A B) or type 'exit': E J

Partially Decrypted Text:

ANS ORTR ITARHR ATARJTOIARTO IRDLTR

Current Key:

G → I
I → N
J → O
K → T
L → J
P → H
R → L
T → S
V → D
X → R
Z → A

Swap plaintext letters (A B) or type 'exit': exit

Conclusion: This experiment shows how monoalphabetic ciphers can be broken using frequency analysis combined with interactive key refinement. By observing partially decrypted text after each refinement, the plaintext can be gradually recovered. The experiment clearly demonstrates the practical weaknesses of monoalphabetic substitution ciphers.

PRACTICAL – 3

Aim: Write a program to implement DES encryption and decryption strategy by taking arguments from user and perform 16 rounds of iterations to get cipher text.

Description: In this experiment, the Data Encryption Standard (DES) algorithm is implemented to perform both encryption and decryption of a given message. The program accepts plaintext and a secret key from the user, processes the data through initial permutation, 16 Feistel rounds, and final permutation to generate ciphertext. The same algorithm, with subkeys applied in reverse order, is used for decryption. This experiment helps in understanding the internal working of DES and its round-based structure.

Theory: DES is a **symmetric-key block cipher** developed by IBM and adopted as a standard in 1977. It operates on **64-bit blocks of data** using a **56-bit effective key** (64 bits including parity). DES follows a **Feistel network structure**, where the plaintext block is divided into two halves and processed through **16 identical rounds**.

Each round involves:

- Expansion of 32-bit data to 48 bits
- XOR operation with a round key
- Substitution using S-boxes
- Permutation using a P-box

The strength of DES comes from **confusion and diffusion**, achieved through substitution and permutation. Although DES is no longer considered secure due to its small key size, it remains important for educational and historical purposes.

Mathematical Concepts:

1. Initial and Final Permutation

$$IP: \{1,2, \dots, 64\} \rightarrow \{1,2, \dots, 64\}$$

2. Feistel Round Structure

$$\begin{aligned} L_i &= R_{i-1} \\ R_i &= L_{i-1} \oplus F(R_{i-1}, K_i) \end{aligned}$$

3. Round Function

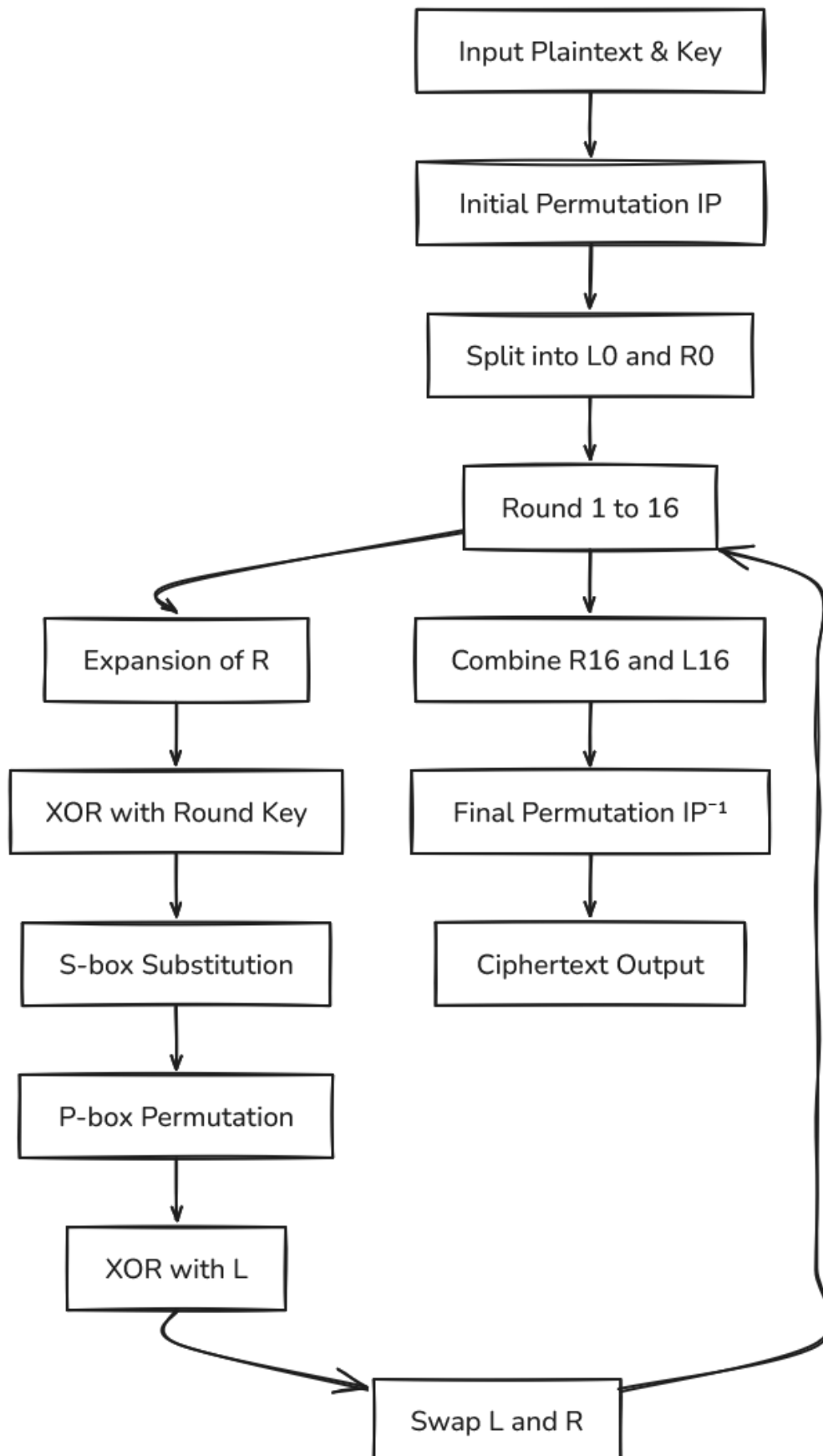
$$F(R, K) = P(S(E(R) \oplus K))$$

Where:

- E = Expansion function (32 $\rightarrow$ 48 bits)
- K = Round key
- S = Substitution using S-boxes
- P = Permutation function

4. Encryption and Decryption

- Encryption: Subkeys applied from K_1 to K_{16}
- Decryption: Subkeys applied from K_{16} to K_1

Flowchart:

Code:

```
def hex2bin(s):
    mp = {'0': "0000", '1': "0001", '2': "0010", '3': "0011",
          '4': "0100", '5': "0101", '6': "0110", '7': "0111",
          '8': "1000", '9': "1001", 'A': "1010", 'B': "1011",
          'C': "1100", 'D': "1101", 'E': "1110", 'F': "1111",
          'a': "1010", 'b': "1011", 'c': "1100", 'd': "1101",
          'e': "1110", 'f': "1111"}
    bin_str = ""
    for i in range(len(s)):
        if s[i] in mp:
            bin_str = bin_str + mp[s[i]]
    return bin_str

def bin2hex(s):
    mp = {"0000": '0', "0001": '1', "0010": '2', "0011": '3',
          "0100": '4', "0101": '5', "0110": '6', "0111": '7',
          "1000": '8', "1001": '9', "1010": 'A', "1011": 'B',
          "1100": 'C', "1101": 'D', "1110": 'E', "1111": 'F'}
    hex_str = ""
    for i in range(0, len(s), 4):
        ch = s[i:i+4]
        if len(ch) == 4 and ch in mp:
            hex_str = hex_str + mp[ch]
    return hex_str

def bin2dec(binary_str):
    decimal = 0
    for i, bit in enumerate(reversed(binary_str)):
        if bit == '1':
            decimal += (1 << i)
    return decimal

def dec2bin(num):
    res = bin(num).replace("0b", "")
    if len(res) % 4 != 0:
        counter = 4 - (len(res) % 4)
        res = '0' * counter + res
    return res

def permute(k, arr, n):
    permutation = ""
    for i in range(len(arr)):
        index = arr[i] - 1
        if index < len(k):
            permutation = permutation + k[index]
        else:
            permutation = permutation + '0'
    return permutation

def shift_left(k, nth_shifts):
    if nth_shifts <= 0:
        return k
    for _ in range(nth_shifts):
        k = k[1:] + k[0]
```

```

    return k
def xor(a, b):
    ans = ""
    min_len = min(len(a), len(b))
    for i in range(min_len):
        if a[i] == b[i]:
            ans = ans + "0"
        else:
            ans = ans + "1"
    return ans
initial_perm = [58, 50, 42, 34, 26, 18, 10, 2,
                60, 52, 44, 36, 28, 20, 12, 4,
                62, 54, 46, 38, 30, 22, 14, 6,
                64, 56, 48, 40, 32, 24, 16, 8,
                57, 49, 41, 33, 25, 17, 9, 1,
                59, 51, 43, 35, 27, 19, 11, 3,
                61, 53, 45, 37, 29, 21, 13, 5,
                63, 55, 47, 39, 31, 23, 15, 7]
exp_d = [32, 1, 2, 3, 4, 5, 4, 5,
         6, 7, 8, 9, 8, 9, 10, 11,
         12, 13, 12, 13, 14, 15, 16, 17,
         16, 17, 18, 19, 20, 21, 20, 21,
         22, 23, 24, 25, 24, 25, 26, 27,
         28, 29, 28, 29, 30, 31, 32, 1]
per = [16, 7, 20, 21,
       29, 12, 28, 17,
       1, 15, 23, 26,
       5, 18, 31, 10,
       2, 8, 24, 14,
       32, 27, 3, 9,
       19, 13, 30, 6,
       22, 11, 4, 25]
sbox = [[14, 4, 13, 1, 2, 15, 11, 8, 3, 10, 6, 12, 5, 9, 0, 7],
        [0, 15, 7, 4, 14, 2, 13, 1, 10, 6, 12, 11, 9, 5, 3, 8],
        [4, 1, 14, 8, 13, 6, 2, 11, 15, 12, 9, 7, 3, 10, 5, 0],
        [15, 12, 8, 2, 4, 9, 1, 7, 5, 11, 3, 14, 10, 0, 6, 13]],
        [[15, 1, 8, 14, 6, 11, 3, 4, 9, 7, 2, 13, 12, 0, 5, 10],
        [3, 13, 4, 7, 15, 2, 8, 14, 12, 0, 1, 10, 6, 9, 11, 5],
        [0, 14, 7, 11, 10, 4, 13, 1, 5, 8, 12, 6, 9, 3, 2, 15],
        [13, 8, 10, 1, 3, 15, 4, 2, 11, 6, 7, 12, 0, 5, 14, 9]],
        [[10, 0, 9, 14, 6, 3, 15, 5, 1, 13, 12, 7, 11, 4, 2, 8],
        [13, 7, 0, 9, 3, 4, 6, 10, 2, 8, 5, 14, 12, 11, 15, 1],
        [13, 6, 4, 9, 8, 15, 3, 0, 11, 1, 2, 12, 5, 10, 14, 7],
        [1, 10, 13, 0, 6, 9, 8, 7, 4, 15, 14, 3, 11, 5, 2, 12]],
        [[7, 13, 14, 3, 0, 6, 9, 10, 1, 2, 8, 5, 11, 12, 4, 15],
        [13, 8, 11, 5, 6, 15, 0, 3, 4, 7, 2, 12, 1, 10, 14, 9],
        [10, 6, 9, 0, 12, 11, 7, 13, 15, 1, 3, 14, 5, 2, 8, 4],
        [3, 15, 0, 6, 10, 1, 13, 8, 9, 4, 5, 11, 12, 7, 2, 14]],
        [[2, 12, 4, 1, 7, 10, 11, 6, 8, 5, 3, 15, 13, 0, 14, 9],
        [14, 11, 2, 12, 4, 7, 13, 1, 5, 0, 15, 10, 3, 9, 8, 6],

```



```

[4, 2, 1, 11, 10, 13, 7, 8, 15, 9, 12, 5, 6, 3, 0, 14],
[11, 8, 12, 7, 1, 14, 2, 13, 6, 15, 0, 9, 10, 4, 5, 3]],
[[12, 1, 10, 15, 9, 2, 6, 8, 0, 13, 3, 4, 14, 7, 5, 11],
[10, 15, 4, 2, 7, 12, 9, 5, 6, 1, 13, 14, 0, 11, 3, 8],
[9, 14, 15, 5, 2, 8, 12, 3, 7, 0, 4, 10, 1, 13, 11, 6],
[4, 3, 2, 12, 9, 5, 15, 10, 11, 14, 1, 7, 6, 0, 8, 13]],
[[4, 11, 2, 14, 15, 0, 8, 13, 3, 12, 9, 7, 5, 10, 6, 1],
[13, 0, 11, 7, 4, 9, 1, 10, 14, 3, 5, 12, 2, 15, 8, 6],
[1, 4, 11, 13, 12, 3, 7, 14, 10, 15, 6, 8, 0, 5, 9, 2],
[6, 11, 13, 8, 1, 4, 10, 7, 9, 5, 0, 15, 14, 2, 3, 12]],
[[13, 2, 8, 4, 6, 15, 11, 1, 10, 9, 3, 14, 5, 0, 12, 7],
[1, 15, 13, 8, 10, 3, 7, 4, 12, 5, 6, 11, 0, 14, 9, 2],
[7, 11, 4, 1, 9, 12, 14, 2, 0, 6, 10, 13, 15, 3, 5, 8],
[2, 1, 14, 7, 4, 10, 8, 13, 15, 12, 9, 0, 3, 5, 6, 11]]]
final_perm = [40, 8, 48, 16, 56, 24, 64, 32,
               39, 7, 47, 15, 55, 23, 63, 31,
               38, 6, 46, 14, 54, 22, 62, 30,
               37, 5, 45, 13, 53, 21, 61, 29,
               36, 4, 44, 12, 52, 20, 60, 28,
               35, 3, 43, 11, 51, 19, 59, 27,
               34, 2, 42, 10, 50, 18, 58, 26,
               33, 1, 41, 9, 49, 17, 57, 25]
keyp = [57, 49, 41, 33, 25, 17, 9,
        1, 58, 50, 42, 34, 26, 18,
        10, 2, 59, 51, 43, 35, 27,
        19, 11, 3, 60, 52, 44, 36,
        63, 55, 47, 39, 31, 23, 15,
        7, 62, 54, 46, 38, 30, 22,
        14, 6, 61, 53, 45, 37, 29,
        21, 13, 5, 28, 20, 12, 4]
shift_table = [1, 1, 2, 2,
               2, 2, 2, 2,
               1, 2, 2, 2,
               2, 2, 2, 1]
key_comp = [14, 17, 11, 24, 1, 5,
            3, 28, 15, 6, 21, 10,
            23, 19, 12, 4, 26, 8,
            16, 7, 27, 20, 13, 2,
            41, 52, 31, 37, 47, 55,
            30, 40, 51, 45, 33, 48,
            44, 49, 39, 56, 34, 53,
            46, 42, 50, 36, 29, 32]
def encrypt(pt, rkb, rk):
    pt = hex2bin(pt)
    pt = permute(pt, initial_perm, 64)
    print("After initial permutation:", bin2hex(pt))
    left = pt[0:32]
    right = pt[32:64]
    for i in range(0, 16):
        right_expanded = permute(right, exp_d, 48)

```

```

    xor_x = xor(right_expanded, rkb[i])
    sbbox_str = ""
    for j in range(0, 8):
        row_str = xor_x[j * 6] + xor_x[j * 6 + 5]
        col_str = xor_x[j * 6 + 1] + xor_x[j * 6 + 2] + xor_x[j * 6 + 3] + xor_x[j
* 6 + 4]
        row = bin2dec(row_str)
        col = bin2dec(col_str)
        val = sbbox[j][row][col]
        sbbox_str = sbbox_str + dec2bin(val)
    sbbox_str = permute(sbbox_str, per, 32)
    result = xor(left, sbbox_str)
    left = result
    if i != 15:
        left, right = right, left
    print(f"Round {i + 1}: L={bin2hex(left)}, R={bin2hex(right)}, Key={rk[i]}")
    combine = left + right
    cipher_text = permute(combine, final_perm, 64)
    return cipher_text

def main():
    plaintext_ascii = input("Enter Plain Text: ").strip()
    key_ascii = input("Enter Key: ").strip()
    pt_hex = plaintext_ascii.encode().hex().upper()
    key_hex = key_ascii.encode().hex().upper()
    pt = pt_hex.ljust(16, '0')[:16]
    key = key_hex.ljust(16, '0')[:16]
    print(f"Plaintext (hex): {pt}")
    print(f"Key (hex): {key}")
    key_bin = hex2bin(key)
    if len(key_bin) < 64:
        key_bin = key_bin.ljust(64, '0')
    elif len(key_bin) > 64:
        key_bin = key_bin[:64]
    key_bin = permute(key_bin, keyp, 56)
    left_key = key_bin[0:28]
    right_key = key_bin[28:56]
    rkb = []
    rk = []
    for i in range(16):
        left_key = shift_left(left_key, shift_table[i])
        right_key = shift_left(right_key, shift_table[i])
        combined_key = left_key + right_key
        round_key_bin = permute(combined_key, key_comp, 48)
        rkb.append(round_key_bin)
        rk.append(bin2hex(round_key_bin))

    print("\n" + "="*50)
    print("ENCRYPTION")
    print("="*50)

```

```

cipher_text_bin = encrypt(pt, rkb, rk)
cipher_text_hex = bin2hex(cipher_text_bin)
print(f"\nCipher Text (hex): {cipher_text_hex}")
print("\n" + "="*50)
print("DECRYPTION")
print("="*50)
rkb_rev = rkb[::-1]
rk_rev = rk[::-1]
plain_text_bin = encrypt(cipher_text_hex, rkb_rev, rk_rev)
plain_text_hex = bin2hex(plain_text_bin)
try:
    plain_text_ascii = bytes.fromhex(plain_text_hex).decode('utf-8',
errors='ignore').rstrip('\x00')
    print(f"Decrypted Text (hex): {plain_text_hex}")
    print(f"Decrypted Text (ASCII): {plain_text_ascii}")
except:
    print(f"Decrypted Text (hex): {plain_text_hex}")
if __name__ == "__main__":
    main()

```

Output:

```

Enter Plain Text: PARUL
Enter Key: 2026
Plaintext (hex): 504152554C000000
Key (hex): 3230323600000000
=====
ENCRYPTION
=====
After initial permutation: 1F0D180A00001004
Round 1: L=00001004, R=11DECB4B, Key=002804452041
Round 2: L=11DECB4B, R=1D3E69D6, Key=0024046082C1
Round 3: L=1D3E69D6, R=F7DC15EF, Key=40040452840B
Round 4: L=F7DC15EF, R=92D92314, Key=4080200E1508
Round 5: L=92D92314, R=4A7E947A, Key=808022087160
Round 6: L=4A7E947A, R=C52B9CDE, Key=A0020260C820
Round 7: L=C52B9CDE, R=2B4C8F90, Key=201200C00C1A
Round 8: L=2B4C8F90, R=BE7107FC, Key=0010508D1218
Round 9: L=BE7107FC, R=BAC0CC97, Key=004050890620
Round 10: L=BAC0CC97, R=D44BF5FA, Key=044110584A04
Round 11: L=D44BF5FA, R=48E15821, Key=06010110409C
Round 12: L=48E15821, R=FCF5990D, Key=0B0001813081
Round 13: L=FCF5990D, R=ABA38265, Key=090088A22221
Round 14: L=ABA38265, R=2E9E3027, Key=100088320B06
Round 15: L=2E9E3027, R=D42A6582, Key=100800140192
Round 16: L=28AF9FFC, R=D42A6582, Key=1028008421C4

Cipher Text (hex): 1C369D7585798997

=====
DECRYPTION
=====
After initial permutation: 28AF9FFCD42A6582

```

Round 1: L=D42A6582, R=2E9E3027, Key=1028008421C4
Round 2: L=2E9E3027, R=ABA38265, Key=100800140192
Round 3: L=ABA38265, R=FCF5990D, Key=100088320B06
Round 4: L=FCF5990D, R=48E15821, Key=090088A22221
Round 5: L=48E15821, R=D44BF5FA, Key=0B0001813081
Round 6: L=D44BF5FA, R=BAC0CC97, Key=06010110409C
Round 7: L=BAC0CC97, R=BE7107FC, Key=044110584A04
Round 8: L=BE7107FC, R=2B4C8F90, Key=004050890620
Round 9: L=2B4C8F90, R=C52B9CDE, Key=0010508D1218
Round 10: L=C52B9CDE, R=4A7E947A, Key=201200C00C1A
Round 11: L=4A7E947A, R=92D92314, Key=A0020260C820
Round 12: L=92D92314, R=F7DC15EF, Key=808022087160
Round 13: L=F7DC15EF, R=1D3E69D6, Key=4080200E1508
Round 14: L=1D3E69D6, R=11DECB4B, Key=40040452840B
Round 15: L=11DECB4B, R=00001004, Key=0024046082C1
Round 16: L=1F0D180A, R=00001004, Key=002804452041
Decrypted Text (hex): 504152554C000000
Decrypted Text (ASCII): PARUL

Conclusion: This experiment successfully demonstrates the working of the DES encryption and decryption algorithm using a 16-round Feistel structure. By implementing each step of DES, including permutations, substitutions, and key scheduling, the experiment provides a clear understanding of how plaintext is transformed into ciphertext and recovered back. Although DES is no longer secure for modern applications, this experiment is useful for learning the fundamentals of block cipher design and symmetric-key cryptography.

PRACTICAL – 4**Aim: Implement RSA cryptosystem.**

Description: In this experiment, the **RSA cryptographic algorithm** is implemented to perform **public-key encryption, decryption, digital signing, and verification**. The program generates a pair of keys: a **public key** used for encryption and verification, and a **private key** used for decryption and signing.

The RSA algorithm first generates two prime numbers, calculates the modulus and Euler's totient, and then computes the public and private exponents. A message is converted into an integer before encryption. Encryption is performed using the public key and modular exponentiation, while decryption is performed using the private key.

Additionally, the experiment demonstrates **digital signatures**, where a message hash is signed using the private key and verified using the public key. This helps students understand how RSA is used for **secure communication and authentication**.

Mathem RSA (**Rivest-Shamir-Adleman**) is an **asymmetric cryptographic algorithm** introduced in **1978**. Unlike symmetric encryption, RSA uses **two different keys**:

- Public Key → Used for encryption
- Private Key → Used for decryption

The RSA algorithm is based on **number theory and modular arithmetic**. Its security relies on the difficulty of **factoring large integers**.

Steps in RSA:

1. Choose two large prime numbers **p** and **q**
2. Compute modulus

$$n = p \times q$$

3. Compute Euler's totient

$$\phi(n) = (p-1)(q-1)$$

4. Choose public exponent **e** such that $1 < e < \phi(n)$ and

$$\gcd(e, \phi(n)) = 1$$

5. Compute private exponent **d**

$$d \equiv e^{-1} \pmod{\phi(n)}$$

Public Key:

$$(n, e)$$

Private Key:

(n,d)

Encryption:

$$C = M^e \bmod n$$

Decryption:

$$M = C^d \bmod n$$

RSA can also be used for **digital signatures** where a message hash is encrypted using the private key and verified using the public key.

Mathematical Concepts:

1. RSA Key Generation

$$n = p \times q$$

$$\phi(n) = (p-1)(q-1)$$

$$d = e^{-1} \bmod \phi(n)$$

2. RSA Encryption

$$C = M^e \bmod n$$

Where:

$M \rightarrow$ Message

$C \rightarrow$ Ciphertext

$e \rightarrow$ Public exponent

$n \rightarrow$ Modulus

3. RSA Decryption

$$M = C^d \bmod n$$

Where:

$d \rightarrow$ Private exponent

4. Digital Signature

Signature generation:

$$S = H(M)^d \mod n$$

Verification:

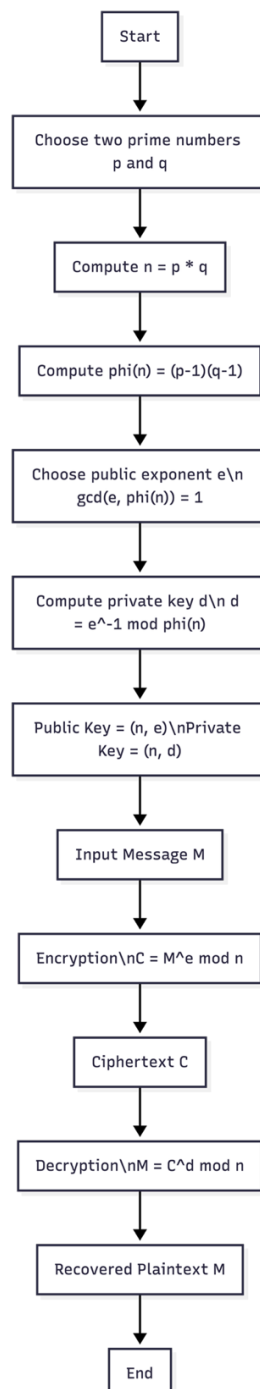
$$H(M) = S^e \mod n$$

Where:

$H(M) \rightarrow$ Hash of message

$S \rightarrow$ Signature

Flowchart:



Code:

```
import random

import math

# function to check prime
def is_prime(num):
    if num < 2:
        return False
    for i in range(2, int(num**0.5)+1):
        if num % i == 0:
            return False
    return True

# generate random prime
def generate_prime():
    while True:
        num = random.randint(100, 500)
        if is_prime(num):
            return num

# modular inverse
def mod_inverse(e, phi):
    for d in range(3, phi):
        if (d * e) % phi == 1:
            return d
    return None

# RSA key generation
def generate_keys():
    p = generate_prime()
    q = generate_prime()
    n = p * q
    phi = (p-1) * (q-1)
```



```
e = 3

while math.gcd(e, phi) != 1:
    e += 2

d = mod_inverse(e, phi)
return ((e, n), (d, n))

# encryption

def encrypt(message, public_key):
    e, n = public_key
    cipher = [(ord(char) ** e) % n for char in message]
    return cipher

# decryption

def decrypt(cipher, private_key):
    d, n = private_key
    plain = [chr((char ** d) % n) for char in cipher]
    return ''.join(plain)

# main program

public_key, private_key = generate_keys()
print("Public Key:", public_key)
print("Private Key:", private_key)
message = input("Enter Message: ")
cipher = encrypt(message, public_key)
print("Encrypted Message:", cipher)
plain = decrypt(cipher, private_key)
print("Decrypted Message:", plain)
```

Output:

Public Key: (7, 14351)

Private Key: (823, 14351)

Enter Message: HELLO

Encrypted Message:

[5460, 1043, 1043, 8921, 643]

Decrypted Message:

HELLO

Conclusion: This experiment demonstrates the working of the **RSA public-key cryptosystem** by generating keys using prime numbers, calculating the modulus and Euler's totient, and applying modular exponentiation for encryption and decryption. The implementation also shows how RSA supports **digital signatures and verification**, ensuring both confidentiality and authentication in secure communication. Although the program uses simplified parameters for educational purposes, it helps in understanding the fundamental mathematical concepts behind **public-key cryptography and secure data transmission**.

PRACTICAL - 5

Aim: Implement Message Integrity using SHA-256 hashing function which creates chain of three blocks. Each block contains index, timestamp, data, previous hash value and current block of hash value. Test message integrity of program by modifying one of the hash value of block.

Description: In this experiment, a **secure HTTPS server** is configured on Kali Linux using **Apache2 and OpenSSL**. The Bash script automates the process of generating a **Root Certificate Authority (CA)**, creating a **server private key**, generating a **Certificate Signing Request (CSR)**, and signing the certificate using the CA.

The script also configures **Apache SSL VirtualHost**, enables SSL modules, and restarts Apache to activate HTTPS. After configuration, the server can be accessed securely using **HTTPS protocol**, ensuring encrypted communication between the client and server.

This experiment demonstrates the working of **SSL/TLS security**, certificate generation, and secure web server configuration.

Theory:

HTTPS (HyperText Transfer Protocol Secure) is an extension of HTTP that uses **SSL/TLS encryption** to secure communication between a client and a web server.

SSL/TLS ensures three main security properties:

- **Confidentiality** – Data is encrypted during transmission.
- **Integrity** – Data cannot be altered during communication.
- **Authentication** – Server identity is verified using digital certificates.

In this experiment, a **Root Certificate Authority (CA)** is created. This CA signs the server certificate, making the certificate trusted by clients that import the CA.

The process involves:

1. Creating a **Root CA key and certificate**
2. Generating a **server private key**
3. Creating a **Certificate Signing Request (CSR)**
4. Signing the certificate using the CA
5. Configuring **Apache SSL VirtualHost**
6. Restarting Apache to enable HTTPS

When a user accesses the server through HTTPS, the browser verifies the certificate and establishes an **encrypted SSL/TLS session**.

Mathematical:

1. Public Key Cryptography

SSL/TLS uses **asymmetric encryption**:

Server generates:

Public Key

Private Key

Certificates contain the **public key**, while the **private key remains secret**.

2. Digital Certificates

A certificate contains:

- Server identity
- Public key
- Issuer (Certificate Authority)
- Validity period
- Digital signature

Certificate signing process:

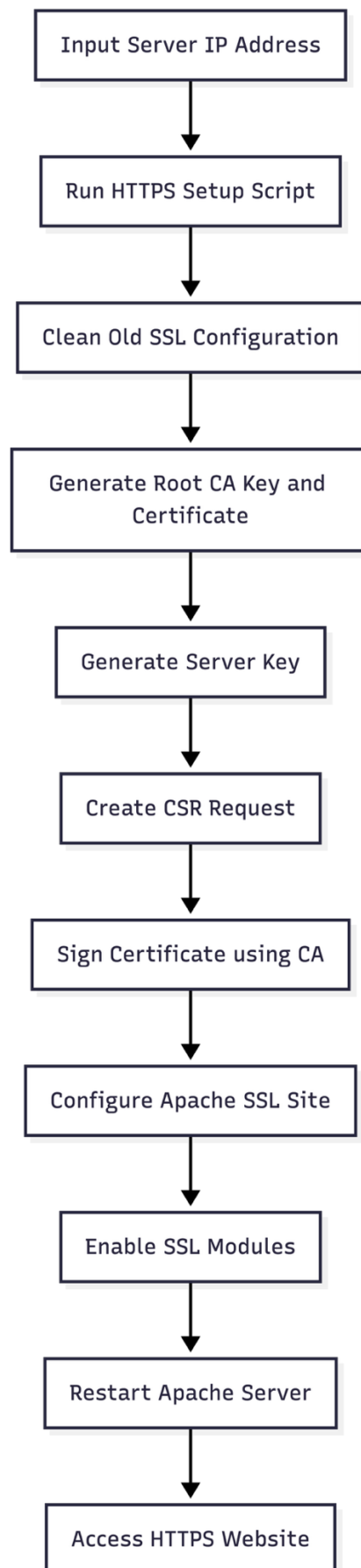
Certificate = $\text{Sign}_{\{CA\}}(\text{PublicKey}, \text{ServerIdentity})$

3. SSL/TLS Handshake

During connection:

1. Client requests HTTPS connection
2. Server sends certificate
3. Client verifies certificate
4. Secure session key is established

This allows encrypted communication.

Flowchart:

Code:

```
#!/bin/bash

set -e

echo "=== Kali Linux HTTPS Lab Setup ==="

echo ""

read -p "Enter your Server IP: " IP

if [ -z "$IP" ]; then

    echo "IP cannot be empty."

    exit 1

fi

CA_DIR="/etc/apache2/ca"

SSL_DIR="/etc/apache2/ssl"

APACHE_SSL_CONF="/etc/apache2/sites-available/default-ssl.conf"

echo ""

echo "[+] Using IP: $IP"

echo "[+] Cleaning old SSL setup..."

sudo rm -rf $CA_DIR

sudo rm -rf $SSL_DIR

sudo a2dissite default-ssl.conf 2>/dev/null || true

sudo mkdir -p $CA_DIR

sudo mkdir -p $SSL_DIR

echo "[+] Creating Root CA..."

sudo openssl genrsa -out $CA_DIR/ca.key 4096

sudo openssl req -x509 -new -nodes \

    -key $CA_DIR/ca.key \

    -sha256 \

    -days 3650 \

    -out $CA_DIR/ca.crt \

    -subj "/C=IN/ST=Gujarat/L=Vadodara/O=KrishSecurity/OU=Lab/CN=Krish-Root-CA"
```

```
echo "[+] Creating Server Key..."

sudo openssl genrsa -out $SSL_DIR/server.key 2048

echo "[+] Creating CSR..."

sudo openssl req -new \
    -key $SSL_DIR/server.key \
    -out $SSL_DIR/server.csr \
    -subj "/C=IN/ST=Gujarat/L=Vadodara/O=KrishSecurity/OU=Lab/CN=$IP"

echo "subjectAltName = IP:$IP" | sudo tee $SSL_DIR/san.cnf >/dev/null

echo "[+] Signing certificate..."

sudo openssl x509 -req \
    -in $SSL_DIR/server.csr \
    -CA $CA_DIR/ca.crt \
    -CAkey $CA_DIR/ca.key \
    -CAcreateserial \
    -out $SSL_DIR/server.crt \
    -days 825 \
    -sha256 \
    -extfile $SSL_DIR/san.cnf

sudo chmod 600 $SSL_DIR/server.key

echo "[+] Enabling SSL modules..."

sudo a2enmod ssl

sudo a2enmod socache_shmcb

echo "[+] Restarting Apache..."

sudo systemctl restart apache2

echo "HTTPS setup complete."
```

Output:

```
=== Kali Linux HTTPS Lab Setup ===

Enter your Server IP: 192.168.1.10

[+] Using IP: 192.168.1.10
```

```
[+] Cleaning old SSL setup...

[+] Creating Root CA...

Generating RSA private key, 4096 bit long modulus
writing new private key to '/etc/apache2/ca/ca.key'

[+] Creating Server Key...

[+] Creating CSR...

[+] Signing certificate...

Certificate request self-signature ok

[+] Enabling SSL modules...

Considering dependency setenvif for ssl:

Module setenvif already enabled

Enabling module ssl.

Enabling module socache_shmcb.

[+] Restarting Apache...

=== HTTPS SETUP COMPLETE ===

Import this CA into Firefox:

/etc/apache2/ca/ca.crt

Then visit:

https://192.168.1.10
```

Conclusion:

This experiment successfully demonstrates how to configure a **secure HTTPS server using Apache and OpenSSL on Kali Linux**. By generating a Root CA, creating server certificates, and enabling SSL modules in Apache, encrypted communication between client and server is established. The experiment helps in understanding **SSL/TLS security, digital certificates, and secure web server configuration**, which are essential components of modern web security.

PRACTICAL - 6**Aim: Implement Digital Signature.**

Theroy: A **Digital Signature** is a cryptographic mechanism used to verify the authenticity and integrity of digital data. It ensures that a message was created by the legitimate sender and was not altered during transmission.

Digital signatures provide three important security properties:

- **Authentication** – verifies the identity of the sender
- **Integrity** – ensures that the message has not been modified
- **Non-repudiation** – prevents the sender from denying the message

The digital signature process typically involves the following steps:

1. Compute a **cryptographic hash** of the message using a hash function such as **SHA-256**.

$$h = H(m)$$

2. The sender uses their **private key** to generate the signature.

$$S = h^d \mod n$$

3. The receiver verifies the signature using the sender's **public key**.

$$v = S^e \mod n$$

4. If

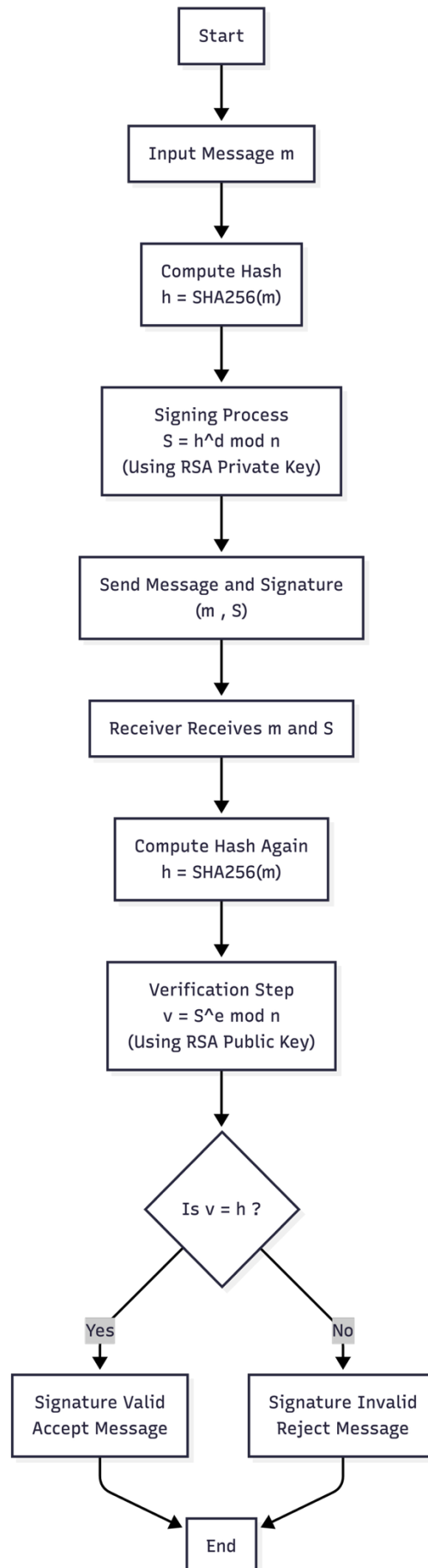
$$v = H(m)$$

then the signature is valid.

Security depends on:

- collision resistance of the hash function
- secrecy of the private key
- computational hardness of RSA

In real systems, standardized schemes such as **RSA-PSS** or **ECDSA** are used instead of textbook RSA.

Flowchart:

Code:

```
#!/usr/bin/env python3
import hashlib
import secrets
# Miller Rabin Prime Test
def is_probable_prime(n, k=8):
    if n < 2:
        return False
    small_primes=[2,3,5,7,11,13,17,19,23,29]
    for p in small_primes:
        if n%p==0:
            return n==p
    d=n-1
    s=0
    while d%2==0:
        d//=2
        s+=1
    for _ in range(k):
        a=secrets.randbelow(n-3)+2
        x=pow(a,d,n)
        if x==1 or x==n-1:
            continue
        composite=True
        for __ in range(s-1):
            x=pow(x,2,n)
            if x==n-1:
                composite=False
                break
        if composite:
            return False
    return True
def generate_prime(bits):
    while True:
        p=secrets.randbits(bits) | 1
        if is_probable_prime(p):
            return p
def egcd(a,b):
    if b==0:
        return (a,1,0)
    g,x,y=egcd(b,a%b)
    return (g,y,x-(a//b)*y)
def modinv(a,m):
    g,x,_=egcd(a,m)
    if g!=1:
        raise ValueError("No inverse")
    return x%m
def generate_rsa_keypair(bits=512):
    p=generate_prime(bits//2)
    q=generate_prime(bits//2)
    n=p*q
    phi=(p-1)*(q-1)
    e=65537
    d=modinv(e,phi)
    return (n,e,d)
```

```
def sha256_int(message):
    h=hashlib.sha256(message.encode()).digest()
    return int.from_bytes(h,'big')
def sign(message,n,d):
    h=sha256_int(message)%n
    signature=pow(h,d,n)
    return signature
def verify(message,signature,n,e):
    h=sha256_int(message)%n
    v=pow(signature,e,n)
    return v==h
print("Generating RSA Keys...")
n,e,d=generate_rsa_keypair()
message=input("Enter message: ")
signature=sign(message,n,d)
print("Signature:",hex(signature))
print("\nVerifying Signature...")
valid=verify(message,signature,n,e)
print("Signature Valid:",valid)
```

Output:

```
Generating RSA Keys...
Enter message: Hello World
Signature: 0x1a4f8b6c7a...
Verifying Signature...
Signature Valid: True
```

Observation:

When the original message and signature are used, verification succeeds.

- Changing the message causes verification failure.
- Changing even a single bit in the signature results in verification failure.
- This demonstrates how digital signatures ensure message integrity and authentication.

Conclusion:

This experiment demonstrates the implementation of a **digital signature scheme using RSA and SHA-256**. The message is first hashed and then signed using the private key. The receiver verifies the signature using the public key. This process ensures message authenticity, integrity, and non-repudiation. The experiment also shows that any modification to the message or signature results in verification failure, illustrating the reliability of digital signatures in secure communication systems.

PRACTICAL - 7

Aim: Study and Configure SSH to authenticate machines, generate session key and transfer files using symmetric key and asymmetric key cryptographic system.

Description: In this experiment, the Secure Shell (SSH) protocol is configured between two machines (client and server). SSH provides a secure communication channel over an insecure network by combining asymmetric cryptography for authentication and symmetric encryption for data transfer.

The experiment includes connecting to a remote server using SSH, generating SSH keys, configuring passwordless authentication, observing the SSH key exchange process using debug mode, and transferring files securely using scp and sftp.

This practical helps students understand how SSH ensures confidentiality, integrity, and authentication in remote communication.

Theory: SSH (**Secure Shell**) is a cryptographic network protocol used for secure communication between two systems over an insecure network. It protects remote login sessions and file transfers by encrypting all transmitted data.

The SSH protocol operates in three main phases:

1. Transport Layer (Key Exchange)

The client and server negotiate cryptographic algorithms and perform a key exchange using algorithms such as Diffie–Hellman or Elliptic Curve Diffie–Hellman. This generates a shared **symmetric session key**.

2. Host Authentication

The server proves its identity using a **host key**. The client stores the server's public host key in the known_hosts file for future verification.

3. User Authentication

The client authenticates using either:

- Password authentication
- Public key authentication

4. Secure Session Communication

After authentication, all communication is encrypted using **symmetric ciphers** such as AES or ChaCha20.

The symmetric session key is **derived during the key exchange process** and is never transmitted directly over the network.

Mathematical Concepts:**1. Asymmetric Cryptography**

SSH uses public/private key pairs for authentication.

Public key → stored on server

Private key → stored on client

Authentication occurs when the client signs a challenge using its private key.

2. Diffie-Hellman Key Exchange

Both client and server compute a shared secret:

Shared\ Secret = $g^{ab} \mod p$

Where

a → client private value

b → server private value

Both sides derive the same session key independently.

3. Symmetric Encryption

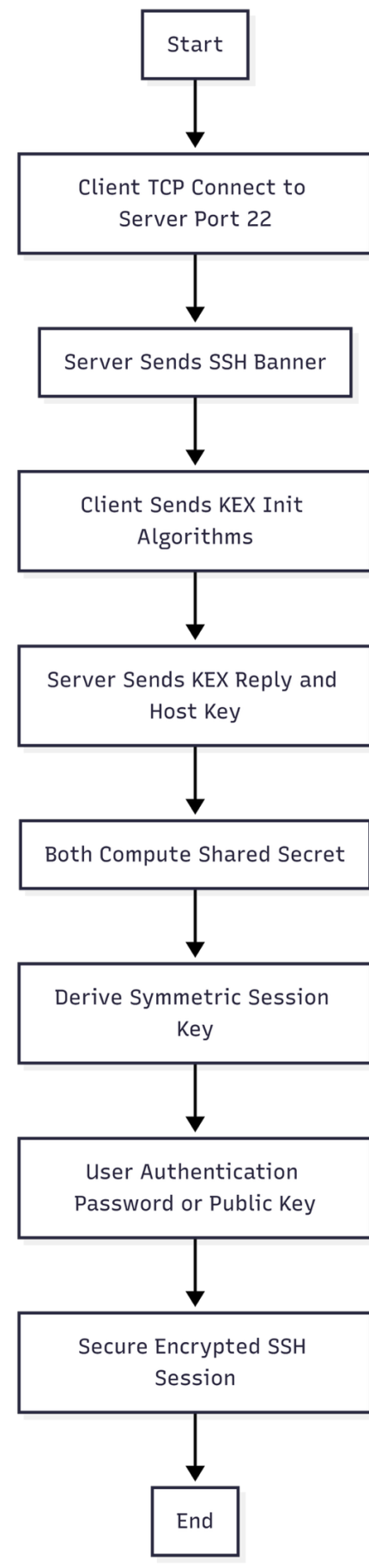
Once the shared key is generated, symmetric algorithms encrypt data.

Examples:

AES-256-GCM

ChaCha20-Poly1305

These algorithms provide **fast encryption for large amounts of data.**

Flowchart:

Code:

Install OpenSSH

sudo apt update

sudo apt install openssh-server openssh-client

Start SSH Service

sudo systemctl enable --now ssh

sudo systemctl status ssh

Connect to Server

ssh bob@192.168.56.102

Generate SSH Key Pair

ssh-keygen -t ed25519 -C "lab_key"

Copy Public Key to Server

ssh-copy-id bob@192.168.56.102

Passwordless SSH Login

ssh bob@192.168.56.102

View SSH Debug Information

ssh -vvv bob@192.168.56.102

Secure File Transfer Using SCP

scp localfile.txt bob@192.168.56.102:~/file.txt

Secure File Transfer Using SFTP

sftp bob@192.168.56.102

put localfile.txt

get file.txt

exit

Output:**SSH Login:**

\$ ssh bob@192.168.56.102

The authenticity of host '192.168.56.102' can't be established.

ED25519 key fingerprint is SHA256:AbC123xyz.

Are you sure you want to continue connecting (yes/no)? yes

Warning: Permanently added '192.168.56.102' to the list of known hosts.

bob@192.168.56.102's password:

SSH Debug Output:

debug1: kex: algorithm: curve25519-sha256

debug1: host key algorithm: ssh-ed25519

debug1: SSH2_MSG_NEWKEYS sent

debug1: SSH2_MSG_NEWKEYS received

debug1: Authentication succeeded (publickey).

SCP File Transfer

```
$ scp report.pdf bob@192.168.56.102:~/
```

```
report.pdf 100% 1.2MB 10.3KB/s 00:01
```

Conclusion: This experiment demonstrates the configuration and working of **OpenSSH for secure remote communication**. By implementing password authentication and public-key authentication, students observe how SSH ensures secure login and file transfer. The experiment also illustrates the **key exchange process that generates symmetric session keys**, enabling efficient encrypted communication between client and server. Through this practical, students gain hands-on experience in configuring, securing, and troubleshooting SSH connections on Linux systems.

Practical - 8

Aim: Implement Deffi Hellmn key exchange algorithm. Generate share secret without Sharing the secret code.

Theory:

The **Diffie–Hellman Key Exchange Algorithm** is a cryptographic protocol that allows two parties to establish a shared secret key over an insecure network.

It was proposed by **Whitfield Diffie and Martin Hellman in 1976** and is widely used in secure communication protocols such as **SSL/TLS, SSH, and VPNs**.

The main idea behind Diffie–Hellman is that both users generate a secret key using publicly exchanged values without sending the actual secret over the network.

Mathematical Process

1. Two public values are chosen:

- A large prime number **p**
- A primitive root **g**

These values are shared publicly.

2. Each user selects a private key.

Krrish chooses **a**

Meet chooses **b**

3. They compute their public keys.

$$A = g^a \bmod p$$

$$B = g^b \bmod p$$

4. They exchange these public keys.
5. Both compute the shared secret key.

Krrish computes:

$$K = B^a \bmod p$$

Meet computes:

$$K = A^b \bmod p$$

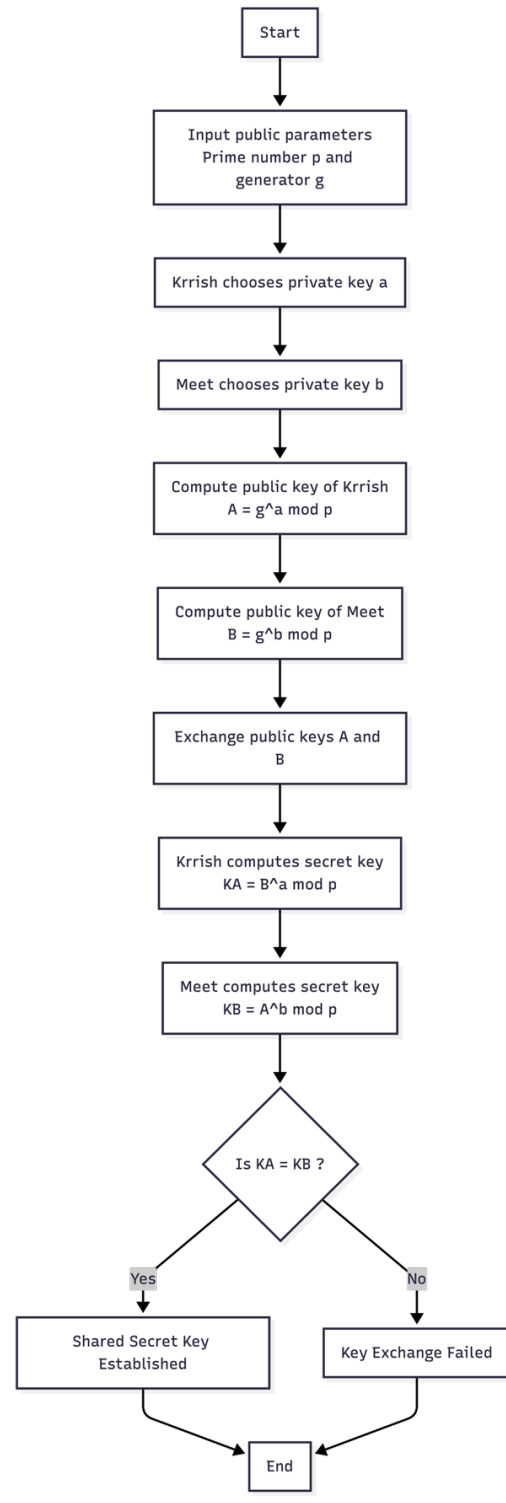
Since

$$B^a \bmod p = A^b \bmod p = g^{ab} \bmod p$$

both users obtain the **same shared secret key**.

The security of Diffie–Hellman relies on the **Discrete Logarithm Problem**, which makes it computationally difficult to determine the private key from the public values.

Flowchart:



Code:

```
p = int(input("Enter prime number (p): "))
g = int(input("Enter primitive root (g): "))
a = int(input("Enter private key of Krrish (a): "))
b = int(input("Enter private key of Meet (b): "))
A = (g ** a) % p
B = (g ** b) % p
print("Public key of Krrish (A):", A)
print("Public key of Meet (B):", B)
KA = (B ** a) % p
KB = (A ** b) % p
print("Secret Key computed by Krrish:", KA)
print("Secret Key computed by Meet:", KB)
```

Output:

```
Enter prime number (p): 23
Enter primitive root (g): 5
Enter private key of Krrish (a): 6
Enter private key of Meet (b): 15
Public key of Krrish (A): 8
Public key of Meet (B): 19
Secret Key computed by Krrish: 2
Secret Key computed by Meet: 2
```

Conclusion:

This experiment demonstrates the **Diffie–Hellman Key Exchange algorithm**, which allows two parties to establish a shared secret over an insecure network without transmitting the secret itself. The protocol relies on modular exponentiation and the computational difficulty of the discrete logarithm problem. Diffie–Hellman forms the basis of many modern secure communication protocols such as **TLS, SSH, and VPN systems**.

PRACTICAL - 9**Aim: Implement ElGamal Cryptosystem.**

Description: In this experiment, the **ElGamal public-key cryptosystem** is implemented using Python. The program generates a **public key and private key** using a prime number and generator in a cyclic group. A message is converted into an integer and encrypted using the public key. The ciphertext consists of two values which are transmitted to the receiver. The receiver uses the private key to compute the shared secret and recover the original message.

The experiment demonstrates the **encryption and decryption process of ElGamal**, which is based on modular exponentiation and the discrete logarithm problem. This practical helps students understand how **public-key cryptography can be used to ensure secure communication**.

Theory: ElGamal is a **public-key cryptosystem** based on the **Discrete Logarithm Problem** in a cyclic group, typically the multiplicative group of integers modulo a prime number.

Key generation involves selecting a **large prime number p**, a **generator g**, and a private key **x**. The public key component **y** is computed as:

$$y = g^x \pmod{p}$$

Public key:

$$(p, g, y)$$

Private key:

$$x$$

Encryption of a message **m** involves choosing a random value **k** and computing two values:

$$c_1 = g^k \pmod{p}$$

$$c_2 = m \cdot y^k \pmod{p}$$

Ciphertext is the pair:

$$(c_1, c_2)$$

Decryption uses the private key **x** to compute a shared secret:

$$s = c_1^x \pmod{p}$$

The original message is recovered by computing:

$$m = c_2 \cdot s^{-1} \pmod{p}$$

The security of ElGamal depends on the difficulty of solving the **Discrete Logarithm Problem**, which makes it computationally hard to derive the private key from the public key.

Mathematical Concept:**1. Key Generation**

$$y = g^x \mod p$$

Where:

$p \rightarrow$ prime number

$g \rightarrow$ generator

$x \rightarrow$ private key

$y \rightarrow$ public key

2. Encryption

$$c_1 = g^k \mod p$$

$$c_2 = m \cdot y^k \mod p$$

Where:

$m \rightarrow$ message

$k \rightarrow$ random number

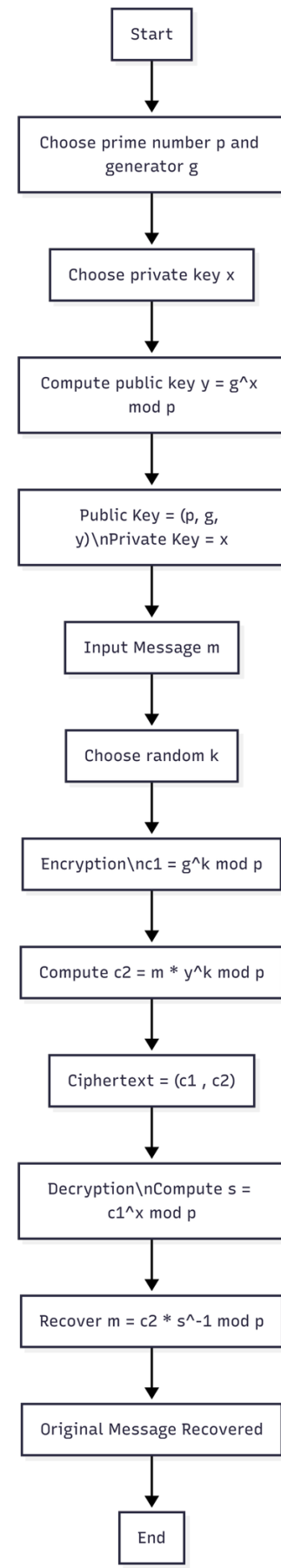
3. Decryption

$$s = c_1^x \mod p$$

$$m = c_2 \cdot s^{-1} \mod p$$

Where:

$s \rightarrow$ shared secret

Flowchart:

Code:

```
import random

# modular inverse

def mod_inverse(a, p):
    return pow(a, -1, p)

# key generation

def generate_keys():
    p = 31847
    g = 5
    x = random.randint(2, p-2)
    y = pow(g, x, p)
    return (p, g, y, x)

# encryption

def encrypt(m, p, g, y):
    k = random.randint(2, p-2)
    c1 = pow(g, k, p)
    c2 = (m * pow(y, k, p)) % p
    return c1, c2

# decryption

def decrypt(c1, c2, p, x):
    s = pow(c1, x, p)
    m = (c2 * mod_inverse(s, p)) % p
    return m

p, g, y, x = generate_keys()
message = int(input("Enter message (number): "))
c1, c2 = encrypt(message, p, g, y)
print("Ciphertext:", c1, c2)
plain = decrypt(c1, c2, p, x)
print("Decrypted Message:", plain)
```


Output:

Enter message (number): 123

Ciphertext: 26587 12345

Decrypted Message: 123

Conclusion:

This experiment successfully demonstrates the working of the ElGamal public-key cryptosystem. By generating keys using a prime number and generator, the algorithm performs secure encryption and decryption using modular arithmetic and the discrete logarithm problem. The practical helps in understanding the principles of public-key cryptography, secure message transmission, and the importance of randomness in encryption systems.

Practical - 10

Aim: Prepare report on any advanced cryptographic system.

Introduction

Homomorphic Encryption (HE) is an advanced cryptographic technique that allows computations to be performed directly on encrypted data without decrypting it. The result of the computation, when decrypted, produces the same result as if the operations were performed on the original plaintext data.

This property allows secure processing of sensitive data in untrusted environments such as **cloud computing platforms**, where the server can perform computations without accessing the actual data.

The concept of Homomorphic Encryption was first proposed by **Rivest, Adleman, and Dertouzos in 1978**, but the first fully functional **Fully Homomorphic Encryption (FHE)** scheme was introduced by **Craig Gentry in 2009**, based on lattice-based cryptography.

Due to increasing concerns about **data privacy, cloud security, and secure AI systems**, Homomorphic Encryption has become a significant area of research in modern cryptography.

Need for Homomorphic Encryption

Traditional encryption protects data:

- **At rest** (stored data)
- **In transit** (data being transmitted)

However, data must usually be **decrypted before processing**, which exposes it to potential security risks.

Homomorphic Encryption solves this problem by allowing:

- Computation directly on encrypted data
- Processing without revealing plaintext
- End-to-end confidentiality of sensitive information

This makes HE highly useful for **cloud computing, machine learning, financial analytics, and healthcare systems**.

Types of Homomorphic Encryption

Homomorphic Encryption schemes are classified based on the operations they support.

1. Partially Homomorphic Encryption (PHE)

Supports only **one type of operation** on encrypted data.

Examples:

RSA Cryptosystem

$$\text{Enc}(m_1) \times \text{Enc}(m_2) = \text{Enc}(m_1 \times m_2)$$

This shows **multiplicative homomorphism**.

Paillier Cryptosystem

$$\text{Enc}(m_1) + \text{Enc}(m_2) = \text{Enc}(m_1 + m_2)$$

This shows **additive homomorphism**.

PHE systems are efficient but limited in functionality.

2. Somewhat Homomorphic Encryption (SHE)

Supports both **addition and multiplication**, but only for a **limited number of operations** before noise grows too large.

These systems are useful for **small computational circuits**.

3. Leveled Homomorphic Encryption (LHE)

Supports computations up to a **fixed circuit depth**.

This type is commonly used in **machine learning inference** where the number of operations is predetermined.

4. Fully Homomorphic Encryption (FHE)

Fully Homomorphic Encryption supports **unlimited additions and multiplications** on encrypted data.

This allows evaluation of **any computable function** without decrypting the data.

Craig Gentry introduced the concept of **bootstrapping**, which reduces noise in ciphertext and enables continuous computation.

Mathematical Foundation

Homomorphic Encryption relies on **lattice-based cryptographic assumptions**, mainly:

Learning With Errors (LWE)

Security is based on solving systems of linear equations containing random noise.

A typical equation is:

$$Ax + e = b$$

Where

$A \rightarrow$ matrix

$x \rightarrow$ secret vector

$e \rightarrow$ small random error

Finding the secret vector x becomes computationally difficult due to the noise.

Ring Learning With Errors (RLWE)

RLWE is a more efficient version of LWE that operates in **polynomial rings**.

It is widely used in modern homomorphic encryption libraries such as:

- Microsoft SEAL
- IBM HELib
- PALISADE

Noise Growth

In homomorphic encryption, ciphertext contains **embedded noise**.

When operations are performed:

Noise increases.

If noise becomes too large, the ciphertext cannot be decrypted correctly.

Bootstrapping is used to **refresh ciphertext and reduce accumulated noise**.

Applications of Homomorphic Encryption

1. Cloud Computing

Users can outsource data processing to cloud servers without revealing sensitive information.

2. Privacy-Preserving Machine Learning

Machine learning models can be trained and evaluated using encrypted data.

3. Secure Database Queries

Encrypted search and analytics can be performed without exposing the original data.

4. Healthcare Systems

Medical records can be analyzed securely without revealing patient information.

5. Financial Services

Banks can perform encrypted financial analytics and fraud detection.

6. Federated Learning

Encrypted gradients can be exchanged securely in distributed machine learning systems.

Advantages

- Provides strong privacy protection
- Data remains encrypted during computation
- Enables secure cloud outsourcing
- Prevents insider threats and data leaks
- Based on post-quantum secure lattice cryptography
- Supports complex encrypted computations

Limitations

- High computational overhead
- Large ciphertext sizes
- Bootstrapping operations are expensive
- Complex parameter selection
- Not suitable for all real-time systems yet

However, ongoing research is improving performance and usability.

Future Trends

- Faster bootstrapping techniques
- Hardware acceleration using GPUs and FPGAs
- Integration with privacy-preserving machine learning
- Standardization by **ISO and NIST**

- Increasing adoption in **healthcare, finance, and cloud computing**

Homomorphic Encryption is expected to become a **key technology for confidential computing and secure AI systems**.

Conclusion

Homomorphic Encryption represents a significant advancement in modern cryptography by allowing computations to be performed directly on encrypted data without revealing the underlying information. This capability enables secure data processing in untrusted environments such as cloud computing platforms. Although current implementations face challenges such as high computational cost and large ciphertext sizes, ongoing research and hardware improvements are making Fully Homomorphic Encryption increasingly practical. As privacy concerns grow in areas such as healthcare, finance, and artificial intelligence, homomorphic encryption is expected to become an essential technology for secure data analytics and confidential computing.

References

1. Craig Gentry, "**Fully Homomorphic Encryption Using Ideal Lattices**", STOC 2009.
2. Microsoft SEAL Documentation – <https://github.com/microsoft/SEAL>
3. IBM HELib Whitepaper – <https://github.com/homenc/HElib>
4. Zvika Brakerski, "**Lattice-Based Cryptography**", Lecture Notes.
5. NIST Post-Quantum Cryptography Project – <https://csrc.nist.gov>